

UNC401 DSA LAB Experiment 9

Q1: Perform the following operations on binary tree:

Tasks:

- a. Write a program to accept an array of integers and create a Binary Tree using level order insertion.
- b. Implement functions to check whether the binary tree is:
 - A Balanced Binary Tree
 - A Complete Binary Tree
 - A Full Binary Tree
 - A Perfect Binary Tree
- c. Perform Inorder, pre-order and postorder traversals of the created binary tree.

CODE:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
};
Node* createNode(int val) {
    Node* temp = new Node();
    temp->data = val;
    temp->left = temp->right = NULL;
    return temp;
}
Node* buildTree(int arr[], int n, int i) {
    if (i >= n)
        return NULL;
    Node* root = createNode(arr[i]);
    root->left = buildTree(arr, n, 2 * i + 1);
    root->right = buildTree(arr, n, 2 * i + 2);
    return root;
}
void inorder(Node* root) {
    if (!root)
        return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
void preorder(Node* root) {
    if (!root)
        return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

```

void postorder(Node* root) {
    if (!root)
        return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
int height(Node* root) {
    if (!root)
        return 0;
    int l = height(root->left);
    int r = height(root->right);
    return (l > r ? l : r) + 1;
}
bool isBalanced(Node* root) {
    if (!root)
        return true;
    int l = height(root->left);
    int r = height(root->right);
    if (abs(l - r) > 1)
        return false;
    return isBalanced(root->left) && isBalanced(root->right);
}
bool isFull(Node* root) {
    if (!root)
        return true;
    if (!root->left && !root->right)
        return true;
    if (root->left && root->right)
        return isFull(root->left) && isFull(root->right);
    return false;
}
int countNodes(Node* root) {
    if (!root)
        return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
bool isPerfect(Node* root, int h, int level = 0) {
    if (!root)
        return true;
    if (!root->left && !root->right)
        return (h == level + 1);
    if (!root->left || !root->right)
        return false;
    return isPerfect(root->left, h, level + 1) &&
        isPerfect(root->right, h, level + 1);
}
bool isComplete(Node* root, int index, int total) {

```

```

    if (!root)
        return true;
    if (index >= total)
        return false;
    return isComplete(root->left, 2 * index + 1, total) &&
        isComplete(root->right, 2 * index + 2, total);
}
int main() {
    int arr[] = {1, 2, 3, 4, 5, 6};
    int n = 6;
    Node* root = buildTree(arr, n, 0);
    cout << "Inorder: ";
    inorder(root);
    cout << "\nPreorder: ";
    preorder(root);
    cout << "\nPostorder: ";
    postorder(root);
    cout << "\nBalanced: " << isBalanced(root);
    cout << "\nFull: " << isFull(root);
    int h = height(root);
    cout << "\nPerfect: " << isPerfect(root, h);
    int total = countNodes(root);
    cout << "\nComplete: " << isComplete(root, 0, total);
    return 0;
}

```

Output

```

Inorder: 4 2 5 1 6 3
Preorder: 1 2 4 5 3 6
Postorder: 4 5 2 6 3 1
Balanced: 1
Full: 0
Perfect: 0
Complete: 1

```

=== Code Execution Successful ===

Q2: Binary Search Tree (BST)

Tasks: You are given a binary search tree on the right side.

- Implement a program to store the given BST in the computer.
 - Search a number in the given BST using input from the user.
 - Perform the deletion operation of a node input by user in a BST.
- Delete a Node from BST: First accept a key K from the user, then

delete the node with the value K from the BST. Ensure that the properties of the BST remain valid after deletion.

- Case 1: Node to be deleted is a leaf node.
- Case 2: Node to be deleted has one child.
- Case 3: Node to be deleted has two children.

CODE:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
};
Node* createNode(int val) {
    Node* temp = new Node();
    temp->data = val;
    temp->left = temp->right = NULL;
    return temp;
}
Node* insert(Node* root, int val) {
    if (!root)
        return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}
bool search(Node* root, int key) {
    if (!root)
        return false;
    if (root->data == key)
        return true;
    if (key < root->data)
        return search(root->left, key);
    return search(root->right, key);
}
Node* findMin(Node* root) {
    while (root->left)
        root = root->left;
    return root;
}
Node* deleteNode(Node* root, int key) {
    if (!root)
        return NULL;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
```

```

else if (key > root->data)
    root->right = deleteNode(root->right, key);
else {
    if (!root->left && !root->right)
        return NULL;
    if (!root->left)
        return root->right;
    if (!root->right)
        return root->left;
    Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

void inorder(Node* root) {
    if (!root)
        return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main() {
    Node* root = NULL;
    int arr[] = {50, 17, 72, 12, 23, 54, 76, 9, 14, 67};
    for (int i = 0; i < 10; i++)
        root = insert(root, arr[i]);
    cout << "Search 23: " << search(root, 23);
    root = deleteNode(root, 17);
    cout << "\nInorder after deletion: ";
    inorder(root);
    return 0;
}

```

Output

Search 23: 1

Inorder after deletion: 9 12 14 23 50 54 67 72 76

=== Code Execution Successful ===

Q3: Implement a max-heap using the following data structures:

- a. Singly Linked List
- b. Binary Search Tree
- c. Array

CODE:

```
#include <iostream>
using namespace std;
struct ListNode {
    int data;
    ListNode* next;
};
ListNode* head = NULL;
void insertList(int val) {
    ListNode* temp = new ListNode();
    temp->data = val;
    temp->next = NULL;
    if (!head || val > head->data) {
        temp->next = head;
        head = temp;
        return;
    }
    ListNode* p = head;
    while (p->next && p->next->data > val)
        p = p->next;
    temp->next = p->next;
    p->next = temp;
}
void deleteMaxList() {
    if (!head)
        return;
    ListNode* temp = head;
    head = head->next;
    delete temp;
}
void printList() {
    ListNode* p = head;
    while (p) {
        cout << p->data << " ";
        p = p->next;
    }
}
struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;
};
BSTNode* createNode(int val) {
```

```

    BSTNode* temp = new BSTNode();
    temp->data = val;
    temp->left = temp->right = NULL;
    return temp;
}

BSTNode* insertBST(BSTNode* root, int val) {
    if (!root)
        return createNode(val);
    if (val < root->data)
        root->left = insertBST(root->left, val);
    else
        root->right = insertBST(root->right, val);
    return root;
}

BSTNode* deleteMaxBST(BSTNode* root) {
    if (!root)
        return NULL;
    if (!root->right) {
        BSTNode* temp = root->left;
        delete root;
        return temp;
    }
    root->right = deleteMaxBST(root->right);
    return root;
}

void inorder(BSTNode* root) {
    if (!root)
        return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int heap[100];
int heapSize = 0;
void swap(int &a, int &b) {
    int t = a;
    a = b;
    b = t;
}

void insertArray(int val) {
    heapSize++;
    heap[heapSize] = val;
    int i = heapSize;
    while (i > 1 && heap[i] > heap[i / 2]) {
        swap(heap[i], heap[i / 2]);
        i /= 2;
    }
}

```

```

void deleteMaxArray() {
    heap[1] = heap[heapSize];
    heapSize--;
    int i = 1;
    while (2 * i <= heapSize) {
        int largest = i;
        if (heap[2 * i] > heap[largest])
            largest = 2 * i;
        if (2 * i + 1 <= heapSize &&
            heap[2 * i + 1] > heap[largest])
            largest = 2 * i + 1;
        if (largest != i) {
            swap(heap[i], heap[largest]);
            i = largest;
        }
        else {
            break;
        }
    }
}

void printArray() {
    for (int i = 1; i <= heapSize; i++)
        cout << heap[i] << " ";
}

int main() {
    cout << "Max Heap using Singly Linked List\n";
    insertList(10);
    insertList(30);
    insertList(20);
    insertList(40);
    printList();
    deleteMaxList();
    cout << "\nAfter delete: ";
    printList();
    cout << "\n\nMax Heap using BST\n";
    BSTNode* root = NULL;
    root = insertBST(root, 10);
    root = insertBST(root, 30);
    root = insertBST(root, 20);
    root = insertBST(root, 40);
    inorder(root);
    root = deleteMaxBST(root);
    cout << "\nAfter delete: ";
    inorder(root);
    cout << "\n\nMax Heap using Array\n";
    insertArray(10);
    insertArray(30);
    insertArray(20);
}

```



```

insertArray(40);
printArray();
deleteMaxArray();
cout << "\nAfter delete: ";
printArray();
return 0;
}

```

Output

Max Heap using Singly Linked List

40 30 20 10

After delete: 30 20 10

Max Heap using BST

10 20 30 40

After delete: 10 20 30

Max Heap using Array

40 30 20 10

After delete: 30 10 20

=== Code Execution Successful ===

Q4: Hash Table Operations:

Tasks:

a. Write a program to implement Hash Table and chaining to avoid collisions and perform the following

functions:

- Insert a Key
- Search a Key
- Delete a Key

b. Write a program to demonstrate the following techniques for avoiding collisions in a Hash table:

- Linear Probing
- Quadratic Probing

c. Write a program to demonstrate Double Hashing.

CODE:

```

#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
}

```

```

};
Node* table[10] = {NULL};
int hashFunc(int key) {
    return key % 10;
}
void insert(int key) {
    int index = hashFunc(key);
    Node* temp = new Node();
    temp->data = key;
    temp->next = table[index];
    table[index] = temp;
}
bool search(int key) {
    int index = hashFunc(key);
    Node* temp = table[index];
    while (temp) {
        if (temp->data == key)
            return true;
        temp = temp->next;
    }
    return false;
}
void deleteKey(int key) {
    int index = hashFunc(key);
    Node* temp = table[index];
    Node* prev = NULL;
    while (temp) {
        if (temp->data == key) {
            if (prev)
                prev->next = temp->next;
            else
                table[index] = temp->next;
            delete temp;
            return;
        }
        prev = temp;
        temp = temp->next;
    }
}
void printChain() {
    for (int i = 0; i < 10; i++) {
        cout << i << " : ";
        Node* temp = table[i];
        while (temp) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
}

```

```

    }
}
int hashLP[10] = {0};
void insertLP(int key) {
    int i = key % 10;
    while (hashLP[i] != 0)
        i = (i + 1) % 10;
    hashLP[i] = key;
}
void printLP() {
    for (int i = 0; i < 10; i++)
        cout << hashLP[i] << " ";
}
int hashQP[10] = {0};
void insertQP(int key) {
    int i = key % 10;
    int j = 1;
    while (hashQP[i] != 0) {
        i = (i + j * j) % 10;
        j++;
    }
    hashQP[i] = key;
}
void printQP() {
    for (int i = 0; i < 10; i++)
        cout << hashQP[i] << " ";
}
int hashDH[10] = {0};
int hash2(int key) {
    return 7 - (key % 7);
}
void insertDH(int key) {
    int i = key % 10;
    int step = hash2(key);
    while (hashDH[i] != 0)
        i = (i + step) % 10;
    hashDH[i] = key;
}
void printDH() {
    for (int i = 0; i < 10; i++)
        cout << hashDH[i] << " ";
}
int main() {
    cout << "HASH TABLE WITH CHAINING\n";
    insert(15);
    insert(25);
    insert(35);
    printChain();
}

```

```

        cout << "\nSearch 25 : ";
        if (search(25))
            cout << "Found";
        else
            cout << "Not Found";
        deleteKey(25);
        cout << "\nAfter Deletion\n";
        printChain();
        cout << "\nLINEAR PROBING\n";
        insertLP(10);
        insertLP(20);
        insertLP(30);
        printLP();
        cout << "\nQUADRATIC PROBING\n";
        insertQP(10);
        insertQP(20);
        insertQP(30);
        printQP();
        cout << "\nDOUBLE HASHING\n";
        insertDH(10);
        insertDH(20);
        insertDH(30);
        printDH();
        return 0;
    }

```

LINEAR PROBING

10 20 30 0 0 0 0 0 0 0

QUADRATIC PROBING

10 20 0 0 0 30 0 0 0 0

DOUBLE HASHING

10 20 0 0 0 30 0 0 0 0

=== Code Execution Successful ===

Output

HASH TABLE WITH CHAINING

0 : NULL
 1 : NULL
 2 : NULL
 3 : NULL
 4 : NULL
 5 : 35 -> 25 -> 15 -> NULL
 6 : NULL
 7 : NULL
 8 : NULL
 9 : NULL

Search 25 : Found

After Deletion

0 : NULL
 1 : NULL
 2 : NULL
 3 : NULL
 4 : NULL
 5 : 35 -> 15 -> NULL
 6 : NULL
 7 : NULL
 8 : NULL
 9 : NULL